PYTHON FOR DATA SCIENCE · LESSON 31 OF 31

Production Python: Logging, Type Hints, Packaging and Code Organisation

Make Python code that other people can run, read and trust. Structure a project into modules, add type hints and check them with mypy, log what a pipeline does without leaking secrets, and package the code so it installs with one command.

Gap-fill lesson | 2026 | Intermediate | 120 min

By the end of this lesson you can

- Organise a script into a package with modules and a single entry point
- Add type hints to functions and explain what a type checker catches
- Configure logging with levels, handlers and a consistent format, and avoid logging secrets
- Describe a minimal pyproject.toml and build an installable package

Datasets used

`telecom_churn.csv`

Why production code is different

A notebook that works on your laptop is a prototype. Production code runs unattended, is read by colleagues months later, and fails in ways you did not expect. Four habits close most of that gap: a clear structure, type hints, logs that tell the story of a run, and packaging so the code installs the same way everywhere.

CASE STUDY **The churn pipeline handover**

An analyst built a churn-scoring pipeline in one notebook. It is now needed by the retention team every week. The engineer taking over needs to understand what it does, see when it fails, install it on a server and change one part without breaking the rest. The notebook cannot do that; a package can.

Step 1: organise the code

A small project has a clear layout:

```
churn_pipeline/
├── pyproject.toml          # how to install the package
├── src/
│   └── churn_pipeline/
│       ├── __init__.py
│       ├── load.py         # reading and checking data
│       ├── features.py     # building model inputs
│       ├── report.py       # summary output
│       └── cli.py          # the entry point people run
└── tests/
    └── test_features.py
```

Each module has one job. `cli.py` contains the only code that reads command-line arguments and calls the others. This is what makes the rest testable: a function in `features.py` can be called from a test with a small table, with no file and no command line.

Step 2: type hints

A **type hint** declares the type a function expects and returns. Python ignores hints at run time, but a **type checker** such as **mypy** reads them and reports mistakes before the code runs.

Python

```python
from typing import Iterable

def tenure_months(signup_month: int, current_month: int) -> int:
    """Months between two month numbers (1 to 12 in the same year)."""
    if not (1 <= signup_month <= 12 and 1 <= current_month <= 12):
        raise ValueError("months must be between 1 and 12")
    return current_month - signup_month

def average(values: Iterable[float]) -> float:
    values = list(values)
    if not values:
        raise ValueError("cannot average an empty sequence")
    return sum(values) / len(values)

print(tenure_months(3, 11), round(average([2.0, 4.0, 9.0]), 2))
```

OUTPUT

```
8 5.0
```

Hints make intent visible: `→ int` says the function returns an integer, and `Iterable[float]` says it accepts any sequence of decimals. A checker flags a call such as `tenure_months("March", 11)` before anyone runs it.

Python's standard library provides the names: `list[str]`, `dict[str, float]`, `int | None` (Python 3.10 and later) and `Optional`-style forms in older code.

Python

```python
# Shown as code, not executed: run `mypy src/` in the project folder.
def load_rows(path: str) → list[dict[str, str]]:
    ...

def churn_rate(churned: int, total: int) → float | None:
    """Return the rate, or None when there are no customers (avoid dividing by zero)."""
    return churned / total if total else None
```

::::tip Hints document, checkers enforce Add hints to public functions first: the inputs and outputs of each module. Run mypy in a continuous-integration job so that a change that breaks a type fails before merging. ::::

Step 3: logging that tells the story

`print` shows a message once. `logging` records messages with a level, a time and the name of the module that wrote them, and sends them to files or other destinations. Configure it once, in the entry point.

Python

```python
import io
import logging

def configure_logging(stream) -> logging.Logger:
    """Set up the root logger once. In production, pass a file path instead of a stream."""
    handler = logging.StreamHandler(stream)
    handler.setFormatter(logging.Formatter("%(asctime)s %(levelname)-7s %(name)s: %
(message)s"))
    root = logging.getLogger()
    root.handlers.clear()
    root.addHandler(handler)
    root.setLevel(logging.INFO)
    return logging.getLogger("churn_pipeline")

buf = io.StringIO()
log = configure_logging(buf)
log.info("run started, source=telecom_churn.csv")
log.info("loaded %d rows, %d columns", 3000, 10)
log.warning("%d rows have a missing total_charges value and were skipped", 2)
log.error("scoring failed for batch %s: %s", "2025-06-01", "model file not found")
print(buf.getvalue())
```

OUTPUT

```
2026-11-11 19:10:58,450 INFO    churn_pipeline: run started, source=telecom_churn.csv
2026-11-11 19:10:58,450 INFO    churn_pipeline: loaded 3000 rows, 10 columns
2026-11-11 19:10:58,450 WARNING churn_pipeline: 2 rows have a missing total_charges
value and were skipped
2026-11-11 19:10:58,451 ERROR   churn_pipeline: scoring failed for batch 2025-06-01:
model file not found
```

Use the logger name to show where a message came from. Use the levels consistently:

Level	Use for	Example
DEBUG	Detail useful only when investigating	"feature matrix shape (3000, 18)"
INFO	Normal progress and summaries	"loaded 3000 rows"
WARNING	Something unexpected that did not stop the run	"2 rows skipped"
ERROR	A failure of one step	"scoring failed"
CRITICAL	The whole run cannot continue	"database unreachable"

::::warn Never log secrets or whole records A log line that prints an API key, a password or a customer's full record becomes a leak as soon as the log is shared. Log counts, IDs that are not sensitive, and error types. If you must log a value to debug, mask it: `key[:4] + "..."`. ::::

Use **%-style arguments** (`log.info("loaded %d rows", n)`) rather than f-strings in log calls. The formatting happens only if the message is actually written, which saves time when a level is turned off.

Step 4: a clear error for bad input

A pipeline should fail early with a message that tells the reader what to fix, not deep inside pandas with a confusing traceback.

Python

```python
import pandas as pd

REQUIRED = ["customer_id", "tenure_months", "monthly_charges", "churn"]

def check_input(df: pd.DataFrame) -> pd.DataFrame:
    missing = [c for c in REQUIRED if c not in df.columns]
    if missing:
        raise ValueError(f"input is missing required columns: {missing}")
    if df["customer_id"].duplicated().any():
        raise ValueError("customer_id must be unique")
    return df

good = pd.DataFrame({"customer_id": ["a", "b"], "tenure_months": [3, 9],
                     "monthly_charges": [29.5, 80.0], "churn": ["No", "Yes"]})
print(check_input(good).shape)

try:
    check_input(good.drop(columns=["churn"]))
except ValueError as e:
    print("rejected:", e)
```

OUTPUT

```
(2, 4)
rejected: input is missing required columns: ['churn']
```

Step 5: organise the entry point

The command-line entry point reads its settings, configures logging, calls the pipeline functions in order and returns an exit code.

Python

```python
# src/churn_pipeline/cli.py  (shown as code; uses the standard library argparse)
import argparse
import logging
import sys

log = logging.getLogger("churn_pipeline")

def main(argv: list[str] | None = None) -> int:
    parser = argparse.ArgumentParser(description="Score customers for churn risk.")
    parser.add_argument("input_csv")
    parser.add_argument("--output", default="scores.csv")
    args = parser.parse_args(argv)
    try:
        # load, features, score and write would be called here
        log.info("finished: %s", args.output)
        return 0
    except Exception:
        log.exception("pipeline failed")   # logs the full traceback at ERROR level
        return 1

if __name__ == "__main__":
    logging.basicConfig(level=logging.INFO)
    sys.exit(main())
```

`main` returns a number, and `sys.exit` passes it to the operating system. Schedulers (Automation lesson) treat 0 as success and anything else as failure.

Step 6: packaging with pyproject.toml

Packaging makes the project installable with `pip install .`, so it can be used on any machine without copying files around. Modern projects describe the package in `pyproject.toml`.

TOML

```toml
# pyproject.toml (shown as text; not executed)
[build-system]
requires = ["setuptools≥68"]
build-backend = "setuptools.build_meta"

[project]
name = "churn-pipeline"
version = "0.1.0"
description = "Score customers for churn risk"
requires-python = "≥3.10"
dependencies = ["pandas≥2.0"]

[project.scripts]
churn-score = "churn_pipeline.cli:main"
```

After `pip install .`, the command `churn-score data.csv` runs the entry point. Version numbers should change when the output changes, so that a report can say which version produced it.

::::tip Test before you package Keep a `tests/` folder with a few tests for `features.py` and `check_input`. Run them with `pytest` before every release. A package that installs but computes wrong values is the worst outcome. ::::

Step 7: a review checklist for production code

- Functions have type hints; mypy passes.
- Each module has one job; `cli.py` is the only place that reads arguments.
- Logging is configured once, uses levels, and never prints secrets or whole records.
- Bad input fails early with a message that names the problem.
- Tests cover the calculations; the package installs in a clean environment.
- The version number and dependency versions are recorded.

::::key What to remember - Structure code into modules with one entry point, so it can be tested and installed. - Type hints document intent; mypy catches mistakes before run time. - Use logging levels; configure logging once; never log secrets. - Validate input early and name the missing column or rule. - Package with `pyproject.toml` and `pip install .` so the code runs the same way everywhere. ::::

Quick quiz

5 questions · tap an answer to check it

1. What does a type checker such as mypy do with `def tenure(m: int) → int` ?

- **A** Runs the function with the right input
- **B** Reports calls and returns that do not match the declared types, before the code runs
- **C** Converts the value to an integer
- **D** Logs each call

Answer: B. Hints are read by checkers; Python itself ignores them at run time.

2. Which logging level fits "11 rows were skipped but the run continued"?

- **A** DEBUG
- **B** INFO
- **C** WARNING
- **D** CRITICAL

Answer: C. Something unexpected happened that did not stop the run.

3. Why use `log.info("loaded %d rows", n)` rather than an f-string?

- **A** f-strings are not allowed in logging
- **B** The message is formatted only when it is written, so disabled levels cost less
- **C** It makes the output bold
- **D** %-style is required by mypy

Answer: B. Lazy formatting avoids work for messages that are never shown.

4. Which of these should never appear in a log?

A The number of rows loaded

B The run start time

C An API key or a password

D The name of the module that logged the message

Answer: C. Logs are often shared and stored for long periods; secrets in them are leaks.

5. What does `main()` returning 1 achieve in a scheduled job?

A It restarts the computer

B The scheduler sees a non-zero exit code and can report the job as failed

C It skips logging

D It prints the number 1 to the report

Answer: B. A non-zero exit code is how command-line programs signal failure.

EXERCISE **Make a script production-ready**

Take one of your own scripts (or the telecom churn example). Organise it into a `src/` package with at least two modules, add type hints to every function, configure logging with INFO and WARNING messages, write a `check_input` function for the required columns, and add a `pyproject.toml` so that `pip install .` works. Run `mypy` and report the errors it finds.

Show solution (worked answer)

Hints: keep the reading code and the calculation code in separate modules. Use `python3 -m venv venv` and `pip install .` to test the install in a clean environment. Fix every mypy error, or add a precise comment explaining each one you leave.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.